

FitBuddy — Codebase Documentation

FitBuddy is a single-page React application (built with Vite) that generates personalized weekly workout plans. Users log in, answer a short onboarding quiz, and receive a plan built from an internal exercise library. Progress, streaks, and saved plans are all stored locally in the browser — there is no backend server.

1.1 What the App Does

- Lets a visitor "log in" with one of two demo accounts.
- Walks them through an 8–9 question onboarding quiz about their fitness goal, experience, schedule, location, equipment, and focus areas.
- Algorithmically generates a personalized weekly workout plan from a 41-exercise internal library based on those answers.
- Lets users browse and instantly apply six pre-built "sample" plans instead of taking the quiz.
- Tracks which workouts have been completed, shows overall plan progress and a day-based streak, and lets users regenerate or edit their plan at any time.

1.2 Key Characteristics

- **Client-only architecture** — everything (auth, preferences, plans, completion history) is computed and stored in the browser; there is no server, API, or database anywhere in the stack.
- **Single algorithmic core** — one function, `generatePlan()`, powers both the quiz flow and the "use a sample plan" flow, so behavior stays consistent no matter how a plan was requested.
- **Per-user data isolation** — even though there's no real backend, `storage.js` scopes every `localStorage` key to the logged-in username, so the two demo accounts never see each other's data in the same browser.
- **Component-driven UI** — pages in `App.jsx` are composed from small, reusable presentational components (cards, buttons, progress bars, step indicators) kept in their own files.
- **Progressive, guided UX** — the quiz is broken into single-question steps with a visible progress indicator, inline validation, and a brief simulated "loading" state before the plan is revealed.

1.3 Intended Use / Audience

This is best understood as a **demo/portfolio-style project** : it showcases a clean, goal-driven onboarding flow, a rules-based recommendation engine, and thoughtful UI polish (custom illustrations, empty states, streaks) — rather than a production fitness product. It would need a real authentication system and a persistent backend before handling genuine user accounts.

1. Project Overview & Tech Stack

Aspect	Detail
Framework	React (function components + hooks)
Build tool	Vite
Routing	react-router-dom
Icons	lucide-react
Persistence	Browser localStorage (no database or API)
Deployment	Vercel (SPA rewrite rule)

The app has no backend. All "accounts," preferences, generated plans, and completed-workout history live entirely in the visitor's browser storage, scoped per logged-in username.

2. Configuration & Entry Files

package.json

Declares the project metadata and scripts:

- dev → runs the Vite development server
- build → produces a production bundle
- preview → serves the production build locally

Dependencies are React, ReactDOM, React Router, Vite, its React plugin, and lucide-react for icons. There are no testing libraries or state-management packages — state is handled with plain useState and localStorage.

index.html

The single HTML shell Vite mounts the app into. It contains:

- A #root div, which is where React attaches the entire application.
- A <script type="module" src="/src/main.jsx"> tag that boots the app.
- A page title ("FitBuddy | Your fitness, your rhythm") and a theme-color meta tag (#073b4c, a dark teal) used for the browser chrome color on mobile.

main.jsx

The actual JavaScript entry point. It:

1. Imports React and createRoot from react-dom/client.

2. Wraps the whole app in `<BrowserRouter>` so that client-side routing (via React Router) works.
3. Renders `<App />` into the `#root` element.
4. Imports the global stylesheet `index.css`.

This is the standard modern React 18 bootstrapping pattern (`createRoot` rather than the older `ReactDOM.render`).

`vercel.json`

A single rewrite rule: every path `(/(.*))` is redirected to `/index.html`. This is required for any client-side-routed single-page app deployed on Vercel — without it, refreshing a deep link like `/dashboard` would 404, because Vercel would look for a physical file at that path instead of letting React Router handle it in the browser.

`_gitignore`

Standard ignore rules for a Node/Vite project (e.g., `node_modules`, build output, editor files) so they aren't committed to version control.

`package-lock.json`

Auto-generated by npm; it pins exact dependency versions so installs are reproducible. Not meant to be edited by hand.

3. App.jsx — The Application Core

This is the largest and most important file. It defines every page/route and most of the interactive logic. It has no separate router configuration file — routes are declared directly at the bottom in the exported `App` component:

```
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/" element={<Home />} />
  <Route path="/dashboard" element={<Dashboard />} />
  <Route path="/create-plan" element={<CreatePlan />} />
  <Route path="/plan" element={<Plan />} />
  <Route path="/sample-plans" element={<Samples />} />
  <Route path="/workout/:id" element={<Workout />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

3.1 Layout

A wrapper component used by every "real" page. Its job is simple but important: **it acts as a route guard**. If `getCurrentUser()` returns nothing (no one is logged in), it renders the `<Login />` screen instead of the requested page's children. If a user is logged in, it renders the `<Navbar />`, the page content, and the `<Footer />`.

3.2 Login

A simple authentication screen (not a route-protected page itself, since it's what protects the others). It:

- Holds username, password, and error in local state.
- On submit, calls `login()` from `storage.js`. If it fails, shows an inline error message.
- On success, navigates to `/create-plan`.
- Displays a hint that two demo accounts exist: `user1` and `user2` (both share the same password, as defined in `storage.js`).

3.3 Home

The public-style marketing/landing content: a hero section with call-to-action buttons ("Create my plan" / "Explore plans"), a decorative illustration built entirely from CSS/SVG shapes (hero-art, WavesIcon, sun/wave divs), a 4-item feature grid (via the small Feature sub-component), and a 4-step "How FitBuddy works" explainer section.

3.4 CreatePlan — the Onboarding Quiz

This is the most complex screen. It renders a **multi-step wizard** using:

- `step` (current question index) and `prefs` (answers collected so far) in state.
- A `fields` array describing eight questions in order: name (text input), goal (single choice), experience level (single choice), sessions per week (single choice), session duration (single choice), workout location (single choice), available equipment (single choice), focus areas (**multi**-choice), and free-text preferences (textarea, optional).
- `choose(value)` updates the answer for the current question. For the `focusAreas` question (marked `multi: true`), it toggles the value in/out of an array instead of overwriting it.
- `next()` validates the current step isn't empty (except the optional preferences step), advances to the next step, or — on the final step — normalizes the data, runs `validatePreferences()`, shows a ~1.1 second "Creating your plan..." loading state, calls `generatePlan()`, saves everything via `storage.js`, and navigates to `/plan`.
- `<StepIndicator>` visually shows quiz progress (see section 5).
- The "Your answers stay in your browser" note reinforces that this is entirely a client-side, `localStorage`-driven app.

3.5 Plan

Displays the currently generated plan in full:

- Redirects to an `<Empty>` state if no plan exists yet.
- Shows a summary strip (Summary sub-component) with Goal, Experience, Sessions/week, Session length, and Equipment.

- Lists every workout for the week as a `<WorkoutCard>`.
- **Regenerate** button calls `generatePlan(prefs, plan)` again (passing the old plan so the generator can detect and avoid producing an identical result — see `planGenerator.js` §7) and shows a temporary "New plan generated" toast.
- **Save plan** button persists the plan permanently to the saved-plans list and returns to the dashboard.

3.6 WeeklyTracker

A checklist-style table (not a route of its own — used inside Dashboard) listing every workout with a toggleable "done" checkbox, the day, workout title, duration, and status ("Ready"/"Completed"). Clicking the checkbox calls `onToggle(workout.id)`.

3.7 Dashboard

The user's home base after they have a plan:

- Redirects to an `<Empty>` state if no plan exists.
- Shows four `<StatCard>` tiles: current goal, sessions/week, number of completed workouts, and overall plan progress percentage.
- Highlights the next incomplete workout ("Up next") with a **Start workout** button linking to `/workout/:id`.
- Shows a `<ProgressBar>` and current streak (via `calculateStreak()`).
- Embeds the WeeklyTracker checklist and a full workout list below it.

3.8 Workout

The detail page for a single workout session, reached via `/workout/:id`:

- Looks up the workout by ID inside the currently saved plan; shows an `<Empty>` "not found" state if it's missing (e.g., stale link).
- Displays a warm-up blurb, then every exercise as a numbered card (name, category, equipment, description, sets/reps/rest).
- A cool-down blurb follows.
- A **"Mark workout complete"** button calls `completeWorkout(id)` and `recordWorkoutDate()` (for streak tracking), then updates its own local done state to show a confirmation.

3.9 Samples

Lists the six curated plans from `samplePlans.js` as `<PlanCard>`s. Clicking **"Use this plan"** builds a synthetic preferences object from that sample's metadata, runs it through the same `generatePlan()` function used by the quiz, saves it, and navigates to the dashboard — meaning sample plans and quiz-built plans share identical underlying logic.

3.10 Empty and NotFound

Small reusable "nothing here yet" screens with a call-to-action button, used whenever the dashboard/plan pages have no data yet, or a workout ID/route

doesn't resolve. NotFound is simply Empty configured for a 404-style message and a link back home.

4. Component Files (src/components/)

These are small, purely presentational (or lightly interactive) building blocks reused across the pages above.

Navbar.jsx

The persistent top navigation bar (rendered by Layout, so it appears on every logged-in page):

- Reads the logged-in user's saved name via `getPreferences()?.name` and greets them ("Hi, {name}") if available.
- Renders links to Home, Dashboard, Create Plan, and Sample Plans using `NavLink` (which automatically applies an active class to the current route).
- Includes a responsive hamburger menu (open state toggled by a button, showing a Menu/X icon from `lucide-react`) for smaller screens.
- Has a **"Create my plan"** call-to-action button and a **"Log out"** link that calls `logout()` from `storage.js` and redirects to `/login`.

Footer.jsx

A simple, static site footer: repeats the brand mark/logo, a tagline, quick links (Home, Dashboard, Create Plan, Sample Plans), and a copyright line.

Button.jsx

A tiny, reusable button abstraction that renders **either** a React Router `<Link>` (if a `to` prop is passed) **or** a native `<button>` (otherwise), so callers don't need to think about which element type they need. Supports a variant class (e.g. `button-quiet`), a type (for form submission), and disabled.

Cards.jsx

Exports three small presentational card components:

- **StatCard** — an icon + label + big value + small detail line, used in the Dashboard's stats grid.
- **WorkoutCard** — shown wherever a workout needs to be listed (Dashboard, Plan page): shows the day, a completed/upcoming status badge, title, duration, exercise count, a "View details" link to the workout page, and — if an `onComplete` callback is supplied — a round checkmark button to mark it done inline.
- **PlanCard** — used on the Sample Plans page: shows a monogram-style "art" tile (initials of the plan name), goal/difficulty, name, description, duration/equipment, and a "Use this plan" button.

ProgressBar.jsx

A minimal horizontal progress bar. Takes a value (0–100) and an optional label boolean; renders a "Weekly progress · N%" caption above a filled bar whose width is set inline via `style={{ width: `${value}%` }}`.

StepIndicator.jsx

Used only inside the CreatePlan quiz. Shows a thin progress track (filled proportionally to $(\text{step} + 1) / \text{total}$) plus a text readout like "Step 03 of 09".

5. Data Files (src/data/)

exercises.js

The exercise library — the raw material every generated plan is built from. It defines a small `exercise(...)` factory function to keep each entry concise, then exports an array of **41 exercises** covering categories: Upper Body, Lower Body, Core, Cardio, Mobility, and Full Body.

Each exercise object has:

- id, name, category
- equipment (e.g., None, Dumbbells, Resistance Bands, Gym)
- difficulty (Beginner / Intermediate / Advanced)
- type (strength, cardio, mobility, stability)
- defaultSets, defaultReps, defaultDuration
- description — a short coaching cue

This is the single source of truth consumed by `planGenerator.js` to build every workout.

samplePlans.js

Six hand-curated "starter" plan **templates** (metadata only — no actual exercise lists baked in): Beginner Full Body, Home Fitness, Strength Foundation, Cardio Starter, Mobility & Flexibility, and Balanced Weekly Fitness. Each has a goal, difficulty, days/duration, equipment, and a short description. When a user picks one on the Samples page, its metadata is converted into a preferences object and fed into the real plan generator — so no two "uses" of a sample are guaranteed to produce byte-identical results (since the generator shuffles the exercise pool).

6. Utility Files (src/utlis/)

storage.js — Persistence Layer

This file is the app's entire "backend." It wraps `localStorage` with safe JSON read/write helpers (read/write, both wrapped in try/catch so a corrupted or

unavailable storage doesn't crash the app).

Key pieces:

- **Hardcoded accounts**: { user1: '123456789', user2: '123456789' } — this is a demo-only, client-side "auth" system with no real security (the password is visible in the shipped JavaScript). It should not be treated as production-grade authentication.
- **login(username, password)** — checks credentials against the hardcoded map and stores the current user under the fitbuddyAuthUser key.
- **logout()** — removes that key.
- **Per-user data scoping** : scopedKey(key) appends the current username (or guest) to every other storage key, so user1 and user2 never see each other's preferences, plans, or completed workouts even though everything lives in the same browser's localStorage.
- **Preferences** : savePreferences / getPreferences — the quiz answers.
- **Plans** : saveCurrentPlan / getCurrentPlan (the single active plan) and savePlan / getSavedPlans (a history list, de-duplicated by plan id).
- **Completion tracking** : getCompletedWorkouts, setWorkoutCompleted (explicit on/off toggle), completeWorkout (idempotent "mark done"). Progress percentages (calculateProgress) are derived by cross-referencing completed workout IDs against the current plan's workout IDs.
- **Streaks** : calculateStreak() reads a list of workout dates (recordWorkoutDate() appends today's date whenever a workout is finished) and counts consecutive days backward from today, stopping as soon as there's a gap greater than one day.

validation.js

A very small helper: validatePreferences(preferences) checks that every required quiz field (name, goal, experience, days, duration, location, equipment, focusAreas) has been filled in, returning an array of the names of any that are missing. CreatePlan uses this as a final safety check before generating a plan, on top of the per-step validation already done as the user clicks through the quiz.

planGenerator.js — The Core Algorithm

This is the "brain" of the app — it turns a preferences object into a full weekly workout plan. Step by step:

1. **Category mapping** — a lookup table (goalCategories) maps each fitness goal (e.g. Strength, Endurance, Flexibility) to the exercise categories that are relevant to it (e.g. Strength → Upper/Lower/Full Body/Core).
2. **Filtering the exercise pool** :
 - equipmentMatches() keeps only exercises usable with the equipment the user selected (special-cased for "No Equipment,"

"Mixed Equipment," and "Full Gym").

- Exercises above the user's stated experience level are excluded via difficultyRank.
 - If the location is "Outdoor," gym/dumbbell exercises are excluded.
 - If filtering leaves fewer than 3 exercises, the filters are progressively relaxed (first dropping the location/category constraint, then falling back to the entire library) so a plan can always be generated.
1. **Prioritization** — the filtered pool is shuffled, then sorted so exercises matching the user's chosen focus areas/goal categories are more likely to appear first.
 2. **Building sessions** :
 - Number of sessions per week is clamped between 2 and 6.
 - Exercises per workout depend on session length: 3 exercises for ≤ 15 min, 4 for ≤ 30 min, 5 for ≤ 45 min, 6 for longer.
 - Each workout picks from the prioritized pool, preferring exercises not already used elsewhere in the week (used Set) and, other than for the very first workout, exercises matching the target categories; it backfills from the general pool if there aren't enough matching exercises left.
 - Every workout is labeled with a weekday (spread evenly across the week) and a title built from its dominant exercise category and the user's goal (e.g. "Upper Body Strength," "Restore & Stretch" for Flexibility goals).
 1. **Anti-repetition check** — a signature string (a concatenation of every exercise ID used) is computed for the new plan. If regenerating produces an identical signature to the previous plan (a possible, if unlikely, outcome of the random shuffle), the first workout's exercise order is simply reversed so the user sees at least some visible change.
 2. The function returns a plan object: id, createdAt timestamp, the signature, the (now-normalized) preferences used, and the full workouts array — this is exactly what gets persisted via saveCurrentPlan/savePlan.

7. How Everything Fits Together (Data Flow)

Login (storage.js: login/getCurrentUser)

CreatePlan quiz validatePreferences() planGenerator.generatePlan(prefs)

exercises.js (source pool)

storage.js: savePreferences / saveCurrentPlan / savePlan

WeeklyTracker / ProgressBar completeWorkout()/recordWorkoutDate()

storage.js: getCompletedWorkouts / calculateProgress / calculateStreak

Samples page short-circuits the quiz: it builds a preferences object directly from a chosen sample's metadata and feeds it into the same generatePlan() used everywhere else, then follows the identical save/dashboard flow.

8. Notable Design Notes & Caveats

- **No real backend or security** — the "login" system is a hardcoded demo (user1/user2, shared password 123456789) intended purely to demonstrate per-user data scoping in localStorage. It is not suitable for real user data.
- **All data is per-browser** — clearing browser storage, using a different browser, or private/incognito mode will reset everything (no plan, no history, no streak).
- **Plan generation is randomized** — because the exercise pool is shuffled before selection, regenerating a plan with identical preferences will usually (not always) produce a different mix of exercises.
- **Responsive navigation** — Navbar.jsx includes its own mobile menu state, independent of routing.
- **SPA routing on Vercel** — the vercel.json rewrite is essential; without it, direct navigation to any non-root URL in production would fail.

9. Conclusion

FitBuddy is a compact but well-structured example of a fully client-side React application. Its architecture cleanly separates concerns across four layers: **pages** (App.jsx), **presentational components** (src/components/), **static data** (src/data/), and **logic/persistence utilities** (src/utils/) — which makes the codebase easy to navigate despite having no backend to lean on.

The most interesting piece of engineering is planGenerator.js: a self-contained recommendation algorithm that filters, prioritizes, and assembles a personalized weekly plan from a shared exercise library, while gracefully falling back when filters are too strict and guarding against generating an identical plan twice in a row. Because both the onboarding quiz and the "sample plans" shortcut route through this same function, the app achieves a lot of apparent variety and personalization from a single, well-designed core.

The trade-off for keeping everything client-side is that persistence and "accounts" are inherently fragile and not secure — data lives only in one browser's localStorage, and the login system is a hardcoded demo rather than real authentication. For anyone looking to extend this project toward production use, the natural next steps would be: replacing the hardcoded accounts with real authentication, moving preferences/plans/history to a backend database, and adding automated tests around planGenerator.js given how central it is to the app's core value proposition.

Overall, FitBuddy demonstrates a thoughtful onboarding UX, a genuinely useful rules-based generation engine, and clean component organization — a solid foundation that could be extended into a full production fitness app with the addition of a real backend.